Objects Revision & Functions Intro

API Data, Destructuring, Equality & Functions — Class Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Run every code snippet in your browser's console as you go, and answer from what **actually printed** — don't guess.

Part 1 — Revision: Working with Real API Data

TRY THIS · BROWSER + CONSOLE

Open this link in your browser:
`https://dummyjson.com/products?select=title,price&limit=10&skip=25`

Copy the **entire response** and store it in a variable named `data`:

```
let data = { ...paste the response here... };
```

Q1. Look at the structure of `data`. Which **key** holds the list of products, and what **keys** does each product have?

Q2. Write the code to print every item with its price in **INR (1 USD = 100 INR)**, **rounded to the nearest whole number**, in this format:

```
{item} : {price in INR}
Example: Honey Jar : 699
```

Hint: `Math.round(1.2345 * 100)` gives `123`.

Q3. Calculate the **total price** of all the items and then print the **INR equivalent** of it (**round it to the nearest integer**).

Q4. Write the code to find the **name** of the item with the **maximum price**.

Part 2 — Destructuring Objects & Arrays

Concept — Destructuring. Destructuring **unpacks** values from an object or an array into **separate variables** in one line, instead of reading them out one at a time.

TRY THIS · CONSOLE

```
const student = { name: "Ajay", age: 20, rollNumber: "2616400901" };
const { name, age } = student;
console.log(name, age);
```

Q5. What printed? How would you print the same two variables **without destructuring**?

TRY THIS · CONSOLE

```
const { rollNumber, city } = student;
console.log(rollNumber, city);
```

Q6. What printed, and why? *(Hint: does `student` have a key called `city`?)*

TRY THIS · CONSOLE

```
const fruits = ["Apple", "Banana", "Mango"];
const [first, second] = fruits;
console.log(first, second);
```

Q7. What printed? What decided which fruit went into `first` and which went into `second`?

TRY THIS · CONSOLE

```
const { n, a } = student;
console.log(n, a);
const [x, y] = fruits;
console.log(x, y);
```

Q8. The array version worked with **any variable names**, but the object version printed `undefined`. Complete the sentence: *object destructuring matches by _____, array destructuring matches by _____.*

TRY THIS · CONSOLE

```
const course = { title: "Web Dev", topics: ["HTML", "CSS"] };
let { title, topics } = course;
title = "Full Stack";
topics.push("JS");
console.log(course);
```

Q9. Did changing `title` change `course.title`? Did `topics.push("JS")` change `course.topics`? Explain the difference using what you learned about **copy by reference** in Class 20.

Concept — Destructuring is NOT a Deep Copy. Destructuring copies **plain values** (strings, numbers, booleans) into new variables. If a value is an **array or object**, the new variable just **points to the same one**, so changing it changes the original too.

Q10. In one sentence: after `let { topics } = course`, are `topics` and `course.topics` **two arrays or one**? How would you check?

Part 3 — Comparing Objects: == and ===

Note: In **Class 19** you compared numbers, strings and booleans with `==` and `===`. Now let's see what happens with **objects** and **arrays**.

TRY THIS · CONSOLE

```
const s1 = { name: "Ajay" };
const s2 = { name: "Ajay" };
const s3 = s1;
console.log(s1 == s2);
console.log(s1 == s3);
console.log(s1 === s2);
console.log(s1 === s3);
```

Q11. What did each line print? `s1` and `s2` look **identical**, so why are they not equal? Why is `s1 === s3` true? *(Think back to copy by reference in Class 20.)*

TRY THIS · CONSOLE

```
console.log([1, 2] === [1, 2]);
console.log([1, 2] == [1, 2]);
```

Q12. Two arrays with the **same values**: are they equal? What does this tell you about what `==` and `===` actually check when both sides are objects or arrays?

Concept — Comparing Objects is Comparing References. When both sides are **objects** (arrays included), `==` and `===` don't compare the **contents**. They are true only if both sides point to the **same object** in memory.

TRY THIS · CONSOLE

```
console.log([5] == 5);
console.log([5] === 5);
console.log([1, 2] == "1,2");
```

Q13. Why does `[5] == 5` give `true` but `[5] === 5` give `false`?

Look it up: Google search **"loose equality in JavaScript"** and open the **MDN docs** page for **Equality (==)**. Read the rules it lists for how `==` compares two values.

Q14. According to MDN, what does `==` do when **(a)** both sides are objects, and **(b)** one side is an object and the other is a primitive?

Look it up: Now Google search **"strict equality in JavaScript"** and open the **MDN docs** page for **Strict equality (===)**. Read its rules.

Q15. According to MDN, when does `===` return `true` for two objects? Which of the two would you trust for comparing objects, and why?

Part 4 — Introducing Functions

TRY THIS · CONSOLE

```
console.log("Honey Jar : " + Math.round(6.99 * 100));
console.log("Ice Cream : " + Math.round(5.49 * 100));
console.log("Kiwi : " + Math.round(2.49 * 100));
```

Q16. What is **repeated** in these three lines, and what **changes** from line to line? If the exchange rate changed from 1 USD = 100 INR to 1 USD = 90 INR, how many places would you have to edit?

Concept — Functions. A **function** is a named block of code that you write **once** and run (**call** / **invoke**) whenever you need it. **Defining** a function only stores it; the code inside runs only when you **call** it.

```
function functionName(parameter1, parameter2) {
  // code to run
}

functionName(argument1, argument2);
```

TRY THIS · CONSOLE

```
function greet() {
  console.log("Hello, welcome to class!");
}
```

Q17. Did anything print? Now run `greet();` three times. What is the difference between **defining** a function and **calling** it?

TRY THIS · CONSOLE

```
function greetStudent(name) {
  console.log(`Hello, ${name}!`);
}
greetStudent("Ajay");
greetStudent("Ravi");
```

Concept — Parameter vs Argument. A **parameter** is the variable name written in the function **definition** (`name`). An **argument** is the actual value you pass in when you **call** it (`"Ajay"`). Each call fills the parameter with that call's argument.

Q18. In the code above, which is the **parameter** and which are the **arguments**? What value does `name` hold during the second call?

Q19. Write a function `printPrice(item, usd)` that prints a line in the Part 1 format `{item} : {price in INR}` (1 USD = 100 INR, rounded). Call it for **Honey Jar** (6.99) and **Kiwi** (2.49). Now, how many places would you edit if the rate changed?

TRY THIS · CONSOLE

```
function add(a, b) {
  console.log(a + b);
}
let result = add(2, 3);
console.log(result);
console.log(add(10, 20) * 2);
```

Q20. What printed for each line? What is stored in `result`, and why did `add(10, 20) * 2` give `NaN`?

Concept — Implicit Return: undefined. If a function has **no** `return` statement, calling it gives back `undefined`. **Printing** a value inside a function (`console.log`) is not the same as **giving it back** to the caller.

TRY THIS · CONSOLE

```
function sum(a, b) {
  return a + b;
}
let answer = sum(2, 3);
console.log(answer);
console.log(sum(10, 20) * 2);
```

Q21. What is stored in `answer` now? Why does `sum(10, 20) * 2` work when `add(10, 20) * 2` did not?

TRY THIS · CONSOLE

```
function check() {
  return "done";
  console.log("Will this print?");
}
console.log(check());
```

Q22. Did `"Will this print?"` appear? What does `return` do to the rest of the function?

Concept — Explicit Return. `return value` sends that value **back** to wherever the function was called, and **immediately ends** the function. The caller can store it in a variable, print it, or use it in a calculation.

Q23. Write a function `toINR(usd)` that **returns** the price in INR (1 USD = 100 INR, rounded). Then rewrite your **Part 1 Q2** loop so it uses `toINR()` to print every item.

Part 5 — Arguments, Parameters & Default Values

TRY THIS · CONSOLE

```
function introduce(name, city) {
  console.log(`${name} lives in ${city}`);
}
introduce("Ajay");
```

Q24. What printed? What value did `city` get when **no argument** was passed for it?

TRY THIS · CONSOLE

```
introduce("Ajay", "Pune", "India");
```

Q25. Was there an error? What happened to the **extra argument** `"India"`?

Concept — Default Values. A parameter can be given a **default value** in the definition (`name = "Guest"`). The default is used whenever that argument is **missing** (or is `undefined`).

TRY THIS · CONSOLE

```
function welcome(name = "Guest") {
  console.log(`Welcome, ${name}!`);
}
welcome("Ravi");
welcome();
```

Q26. What printed for each call? When was the **default value** used?

Q27. Predict, then check: what do `welcome(undefined)` and `welcome(null)` print? Why are they different?